ROS2 Parte 5: Pick and Place, MoveIt Setup Assistant y Planificación de Trayectorias para el UR5 con Gripper

Daniel Felipe Escobar Castellanos*, Juan Diego García Molina*
*Facultad de Ingeniería Electrónica
Escuela Colombiana de Ingeniería Julio Garavito
Curso: Robótica (RBTC) — Profesor: Daniel Palomino Suárez

***Resumen*—Se documenta el desarrollo de una aplicación de manipulación robótica basada en ROS 2 Kilted, Gazebo Sim y MoveIt 2 para un manipulador Universal Robots UR5 equipado con un gripper paralelo. Se estructuró un espacio de trabajo con paquetes de descripción, simulación, configuración de movimiento y control personalizado. Se implementó un nodo en C++ que utiliza `MoveGroupInterface` para planificar y ejecutar una secuencia de configuraciones articulares. Asimismo, se incorporó una propuesta de rutina de *Pick and Place* con apertura y cierre del gripper y señales digitales simuladas. El proceso incluyó la corrección de problemas de descripción URDF/Xacro, controladores, límites articulares y colisiones con el plano del suelo. Las evidencias muestran la carga de los modelos en Gazebo y RViz, el uso de MoveIt Setup Assistant, un intento fallido y la posterior ejecución satisfactoria de trayectorias. Se distingue entre las pruebas verificadas y las funcionalidades que requieren evidencia adicional.**

***Index Terms*—ROS 2, UR5, Gazebo Sim, MoveIt 2, Pick and Place, planificación de trayectorias, `ros2_control`, gripper.**

I. Introducción

Los sistemas de manipulación robótica necesitan integrar modelos cinemáticos, planificación de movimientos y control de actuadores para ejecutar tareas de forma reproducible. En este laboratorio se utilizó el ecosistema ROS 2 junto con Gazebo Sim y MoveIt 2 para construir una aplicación sobre el robot UR5 con gripper paralelo. Gazebo permite observar el comportamiento del robot simulado, mientras que MoveIt gestiona objetivos de movimiento, planificación y validación de trayectorias [1], [3], [2].

La actividad contempla el control manual del manipulador y del gripper, la programación de una secuencia *Pick and Place* con varios puntos, el empleo de entradas y salidas digitales simuladas y la generación de una configuración de MoveIt mediante *Setup Assistant*. Se utilizaron paquetes del proyecto base [5] y se desarrollaron componentes propios para ampliar sus funcionalidades.

II. Objetivos

II-A. Objetivo general

Integrar el UR5 con gripper en ROS 2, Gazebo Sim y MoveIt 2 para planificar y ejecutar desplazamientos articulares y estructurar una rutina automatizada de *Pick and Place*.

Tabla I: Componentes principales de la implementación

Componente	Función
ROS 2 Kilted	Comunicación entre nodos, tópicos, parámetros y acciones.
Gazebo Sim	Simulación del UR5, gripper y entorno.
`ros2_control`	Interfaz de control de articulaciones simuladas.
MoveIt 2	Modelo semántico y planificación de trayectorias.
OMPL	Generación de trayectorias articulares.
RViz	Visualización y solicitud interactiva de movimientos.
C++/Python	Implementación de nodos y archivos *launch*.

II-B. Objetivos específicos

- Preparar y compilar los paquetes de descripción y simulación del UR5 con gripper.
- Identificar los tópicos, acciones y controladores utilizados para gobernar el brazo y el efector final.
- Crear una configuración de MoveIt 2 con el grupo cinemático `manipulator` y sus estados almacenados.
- Implementar un nodo C++ que invoque `MoveGroupInterface` para planificar y ejecutar desplazamientos entre varios objetivos.
- Incorporar comandos del gripper y señales digitales simuladas a la lógica de la rutina.
- Documentar las dificultades, sus soluciones y las evidencias observadas.

III. Entorno y organización del proyecto

El desarrollo se llevó a cabo con ROS 2 Kilted y Gazebo Sim, mediante el *workspace* `~/workspaces/ur5_ws`. Adicionalmente, se utilizó un entorno ROS 2 previamente compilado en `~/ur5_ws`, que proporciona paquetes de Universal Robots.

Los paquetes principales se organizaron como sigue:

```
ur5_ws/
  src/
    ur5_gripper_packages/
      ur5_robotiq_description/
      ur5_robotiq_bringup/
    ur5_gripper_moveit_config/
    ur5_pick_place/
    ur5_moveit_control/
```

Tabla II: Objetivos articulares utilizados para las pruebas (rad)

Estado	q_1	q_2	q_3	q_4	q_5	q_6
HOME	0	-1.57	1.57	-1.57	-1.57	0
PICK	0.5	-1.2	1.3	-1.5	-1.57	0.5
TRANSFER	0	-1.3	1.4	-1.5	-1.57	0
PLACE	-0.5	-1.2	1.3	-1.5	-1.57	-0.5

```
      src/ur5_moveit_node.cpp
      launch/
        ur5_moveit.launch.py
        moveit_gazebo.launch.py
        ur5_moveit_gazebo_node.launch.py
```

Listing 1: Organización conceptual del espacio de trabajo.

Para cargar los entornos se definió el script `activar_entorno.sh`, que ejecuta en orden `source` sobre la instalación de Kilted, el espacio de trabajo de Universal Robots y el espacio de trabajo del laboratorio.

IV. Fundamentos del sistema

IV-A. *Descripción cinemática y control*

El modelo URDF/Xacro define los enlaces, las articulaciones y las geometrías necesarias para interpretar los movimientos del UR5. El modelo semántico SRDF, generado para MoveIt, describe los grupos de planificación y los estados articulares definidos por el usuario. En esta actividad se trabajó principalmente con el grupo `manipulator`, correspondiente a las seis articulaciones del brazo.

La ejecución sobre el simulador se apoya en `gz_ros2_control` y en el controlador `scaled_joint_trajectory_controller`. La planificación y la ejecución son procesos diferentes: una trayectoria puede ser generada correctamente y, aun así, no llegar a ejecutarse si falla el controlador o si la configuración de seguridad no es adecuada [4], [3].

IV-B. *Objetivos articulares*

El estado del manipulador se representa mediante el vector

$$\boldsymbol{q} = \begin{bmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 \end{bmatrix}^T, \tag{1}$$

donde cada q_i expresa el ángulo de una articulación en radianes. Para cada par de configuraciones $\boldsymbol{q}_i$ y $\boldsymbol{q}_f$, MoveIt busca una trayectoria factible $\boldsymbol{q}(t)$, compatible con los límites articulares y las colisiones representadas en su escena de planificación.

Los valores de la Tabla II constituyen objetivos de movimiento, no posiciones cartesianas calibradas respecto a un objeto físico. Su empleo requiere validar las trayectorias en la escena de planificación y verificar que no interfieran con el suelo del simulador.

V. Desarrollo experimental

V-A. *Carga y visualización del robot*

Se inició el modelo del UR5 con gripper desde el paquete de simulación y se verificó su representación mediante Gazebo y RViz. La Fig. 1 muestra ambos entornos ejecutándose simultáneamente, con el robot visible en cada ventana.

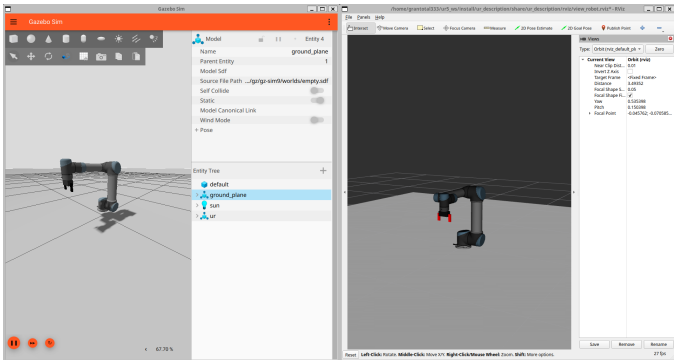


Figura 1: Visualización simultánea del UR5 con gripper en Gazebo Sim y RViz durante la preparación de la simulación.

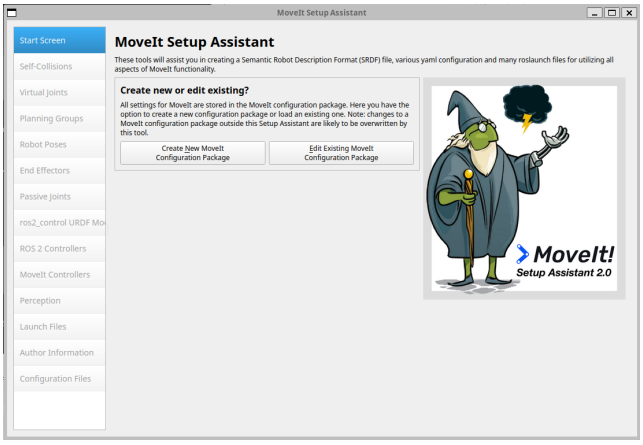


Figura 2: Pantalla de inicio de MoveIt Setup Assistant empleada para configurar el manipulador UR5 con gripper.

V-B. *Generación de la configuración MoveIt*

Mediante *MoveIt Setup Assistant* se preparó el paquete `ur5_gripper_moveit_config`, incorporando la descripción semántica del manipulador, grupos de planificación, poses y parámetros de movimiento. La Fig. 2 registra la herramienta utilizada.

Se construyó una descripción de demostración que incorpora `mock_components/GenericSystem` para comprobar la planificación en RViz sin depender de Gazebo. Para la integración posterior se empleó un Xacro de planificación independiente, mientras que Gazebo mantuvo el modelo con `gz_ros2_control`.

V-C. *Comunicación y controladores*

En Gazebo se verificó la actividad de `joint_state_broadcaster`, `scaled_joint_trajectory_controller` y `gripper_controller`. El estado del robot se consultó desde `/joint_states`, publicado por el *broadcaster*. El controlador del brazo dispone de una acción `FollowJointTrajectory`, que ofrece el mecanismo de ejecución de trayectorias planificadas.

```
source ~/workspaces/ur5_ws/activar_entorno.sh
ros2 control list_controllers
ros2 topic echo /joint_states --once
ros2 action list -t
```

Listing 2: Verificación de controladores y estados.

V-D. *Integración de MoveIt y Gazebo*

Se implementaron archivos *launch* separados: `moveit_gazebo.launch.py` inicia los componentes de planificación y visualización sin crear otro `controller_manager`; `ur5_moveit_gazebo_node.launch.py` proporciona los parámetros necesarios al nodo de control personalizado. La sincronización con el simulador se estableció mediante `use_sim_time=true` y la publicación `/clock`.

```
# Terminal 1
ros2 launch ur5_robotiq_bringup \
  sim_gz_sgb.launch.py launch_rviz:=false

# Terminal 2
ros2 launch ur5_moveit_control \
  moveit_gazebo.launch.py
```

Listing 3: Inicio de Gazebo y del entorno de planificación.

V-E. *Nodo C++ de planificación y ejecución*

Se desarrolló `ur5_moveit_node.cpp` utilizando la clase `MoveGroupInterface`. El nodo configura el grupo `manipulator`, establece objetivos articulares, solicita la planificación con OMPL y comprueba el resultado antes de solicitar la ejecución. Un ejecutor ROS 2 mantiene activos los mecanismos de comunicación durante las llamadas a MoveIt.

```
move_group.setStartStateToCurrentState();
move_group.setJointValueTarget(target);
MoveGroupInterface::Plan plan;
auto planning_result = move_group.plan(plan);
if (planning_result ==
    moveit::core::MoveItErrorCode::SUCCESS) {
    move_group.execute(plan);
}
```

Listing 4: Secuencia esencial del control mediante MoveIt.

En las pruebas del nodo se confirmó, mediante los registros, la finalización de las etapas HOME, PICK, PLACE y HOME. Se emplearon escalas conservadoras de velocidad y aceleración de 0,10.

V-F. *Lógica propuesta para Pick and Place*

La versión ampliada de la aplicación define la secuencia HOME → PICK → TRANSFER → PLACE → HOME. Tras HOME se ordena abrir el gripper; en PICK se solicita su cierre; al llegar a TRANSFER se espera una entrada digital simulada; al finalizar PLACE se ordena abrir el gripper y se activa una salida digital.

Para el gripper se definieron referencias de apertura `[0.04, 0.04]` y cierre `[0.004, 0.004]`. Estas órdenes no demuestran por sí solas que se haya sujetado un objeto: para ello se requiere verificar la respuesta de las articulaciones y, en una simulación de agarre físico, el contacto con el objeto.

Tabla III: Interfaces ROS 2 de la rutina integrada

Tópico	Tipo de mensaje
`/gripper_controller/commands`	`Float64MultiArray`
`/pick_place/digital_input`	`Bool`
`/pick_place/digital_output`	`Bool`
`/pick_place/status`	`String`

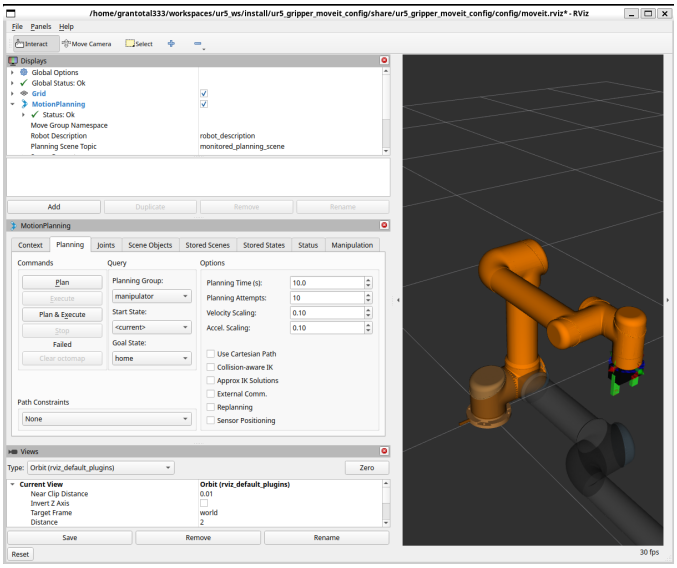


Figura 3: Intento de planificación marcado como `Failed` en RViz durante la depuración de la configuración del robot y su escena de colisiones.

```
ros2 topic pub --once \
  /pick_place/digital_input \
  std_msgs/msg/Bool '{data: true}'
```

Listing 5: Activación manual de la entrada digital simulada.

VI. Resultados y análisis

VI-A. *Diagnóstico del fallo inicial*

En un primer intento, el panel MotionPlanning indicó `Failed` (Fig. 3). Los registros del sistema permitieron identificar que el estado inicial producía contactos con `ground_plane`, incluyendo los enlaces `gripper_base_link`, `left_finger_link`, `right_finger_link`, `wrist_2_link` y `wrist_3_link`. El planificador detuvo la solicitud en la comprobación de colisiones del estado inicial.

Para resolver la inclusión incorrecta del suelo como parte del modelo del robot en MoveIt, se separaron la descripción de simulación y la descripción cinemática empleada para planificar. Este cambio permitió continuar las pruebas sin tratar el suelo como enlace del manipulador. Es necesario aclarar que retirar dicho enlace no equivale a incluir el suelo real de Gazebo como obstáculo en la escena de MoveIt; esa representación debe incorporarse si se requiere planificación con prevención explícita de colisiones contra el piso.

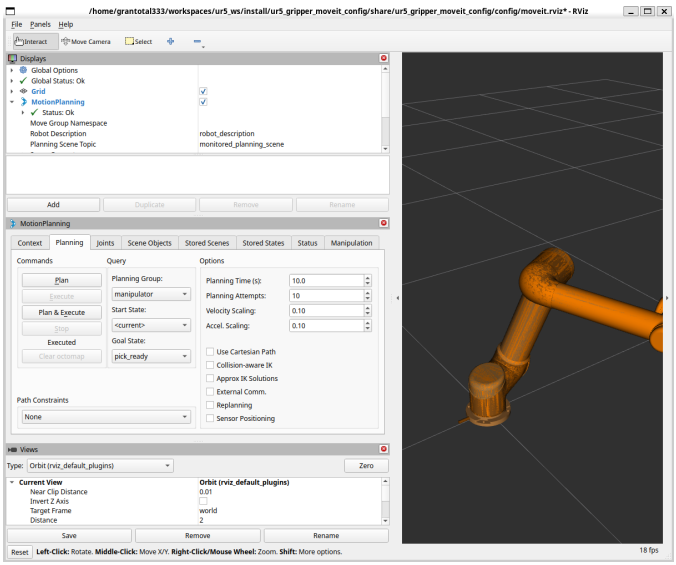


Figura 4: Trayectoria hacia la pose `pick_ready` ejecutada satisfactoriamente en el panel MotionPlanning de RViz.

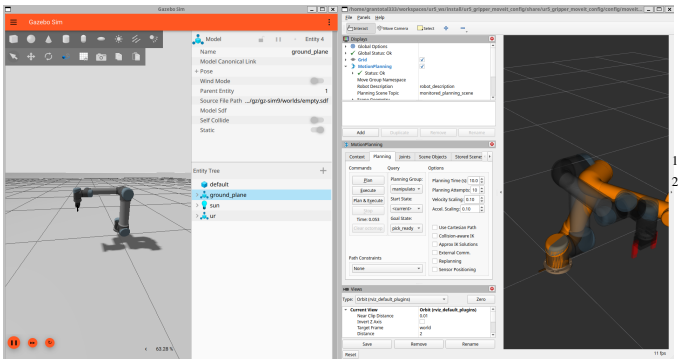


Figura 5: Gazebo y RViz durante una prueba de planificación; el panel muestra un tiempo de planificación aproximado de 0.053 s.

VI-B. Planificación y ejecución satisfactoria

La Fig. 4 registra el estado `Executed` mostrado en RViz para una trayectoria hacia `pick_ready`. La Fig. 5 muestra otra visualización de Gazebo y RViz con una trayectoria planificada y el indicador del tiempo de planificación.

Adicionalmente, los registros del nodo C++ mostraron planificación y ejecución exitosas para HOME, PICK, PLACE y HOME con tiempos de generación de planes del orden de centésimas de segundo. Estos tiempos corresponden a la etapa de planificación, no a la duración total del movimiento.

VI-C. Interpretación de las evidencias

Las fotografías permiten verificar la representación del robot en ambos visualizadores, el uso de Setup Assistant, la presencia de una planificación fallida y un estado posterior de ejecución exitosa. La comparación muestra un avance desde el diagnóstico de errores hasta la operación del entorno de planificación. No obstante, estas imágenes no constituyen evidencia suficiente de agarre físico de objetos ni de una ejecución integral con entrada y salida digitales.

Tabla IV: Tiempos de planificación registrados por el nodo C++

Movimiento	Planificación (s)	Ejecución
HOME inicial	0.0307	Exitosa
PICK	0.0255	Exitosa
PLACE	0.0153	Exitosa
HOME final	0.0190	Exitosa

VII. Dificultades encontradas y soluciones

VIII. Reproducibilidad

Para reproducir el sistema en un equipo compatible se requiere instalar ROS 2, Gazebo, los paquetes de Universal Robots, MoveIt 2 y las dependencias del proyecto. Los paquetes fuente se encuentran en el repositorio del laboratorio [6].

```
cd ~/workspaces/ur5_ws
source /opt/ros/kilted/setup.bash
source ~/ur5_ws/install/setup.bash
colcon build --symlink-install
source install/setup.bash
```

Listing 6: Compilación y carga del entorno.

Una vez que Gazebo y MoveIt estén activos, el nodo de control puede iniciarse con:

```
ros2 launch ur5_moveit_control \
  ur5_moveit_gazebo_node.launch.py
```

Listing 7: Ejecución del nodo de control mediante MoveIt.

La reconstrucción requiere contar con los paquetes del espacio de trabajo base y con las dependencias externas indicadas en los archivos `package.xml`. Las carpetas `build/`, `install/` y `log/` no se consideran código fuente y se regeneran mediante `colcon`.

IX. Conclusiones

La práctica permitió integrar un modelo UR5 con gripper dentro del ecosistema ROS 2, relacionando los estados articulares del simulador con las funcionalidades de planificación de MoveIt 2. Se verificó el funcionamiento de los controladores, la disponibilidad de la descripción cinemática y la ejecución de trayectorias desde RViz y desde un nodo C++ propio.

La depuración mostró que una configuración de planificación puede fallar aun cuando los tópicos y controladores se encuentren activos. En particular, la presencia de `ground_plane` dentro de la descripción del robot ocasionaba contactos en el estado inicial; separar correctamente la descripción de planificación y simulación permitió continuar las pruebas.

La lógica de *Pick and Place* propuesta incorpora movimientos articulares, control del gripper, estados informativos y señales digitales simuladas. Para cerrar experimentalmente esta actividad se debe adjuntar evidencia de la ejecución completa de la secuencia y del comportamiento efectivo del gripper. También es recomendable representar los obstáculos físicos de Gazebo dentro de la escena de planificación de MoveIt para evaluar seguridad de las trayectorias.

Tabla V: Resumen de dificultades técnicas encontradas durante el desarrollo

Dificultad	**Diagnóstico**	**Acción aplicada**
Paquete MoveIt no encontrado	No se había cargado el *overlay* del laboratorio.	Cargar `/opt/ros/kilted`, `~/ur5_ws` y `~/workspaces/ur5_ws`.
Archivo `.setup_assistant` inválido	Referencia vacía del paquete de descripción y ajustes de formato YAML.	Establecer el paquete y la ruta Xacro correspondientes; corregir la sintaxis.
Ausencia de estados articulares en el demo	La descripción inicial no incluía `ros2_control`.	Crear un Xacro de demostración con `GenericSystem`.
Fallos de parametrización temporal	Faltaban límites de aceleración en articulaciones del brazo.	Establecer límites de aceleración en `joint_limits.yaml`.
Planificación fallida por colisiones	El modelo de MoveIt incluía `ground_plane` como enlace que entraba en contacto con el robot.	Separar la descripción de planificación del modelo de Gazebo.
Riesgo de controladores duplicados	El *launch* demo crea componentes de control independientes.	Utilizar un *launch* de MoveIt sin segundo `controller_manager`.

Repositorio y evidencias

El código fuente del laboratorio se centraliza en:

https://github.com/grantotal333/Lab_4_Robotica

Referencias

[1] Open Robotics, "ROS 2 Documentation," [En línea]. Disponible: https://docs.ros.org/

[2] Open Robotics, "Gazebo Documentation," [En línea]. Disponible: https://gazebosim.org/docs/

[3] MoveIt, "MoveIt 2 Documentation," [En línea]. Disponible: https://moveit.picknik.ai/

[4] ROS Controls, "ros2_control Documentation," [En línea]. Disponible: https://control.ros.org/

[5] D. Palomino S., "ur5_gripper_packages," GitHub. [En línea]. Disponible: https://github.com/DanielPalominoS/ur5_gripper_packages

[6] J. D. García Molina y D. F. Escobar Castellanos, "Lab_4_Robotica," GitHub. [En línea]. Disponible: https://github.com/grantotal333/Lab_4_Robotica